

Q.195 | LC 1438: Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit

Target Companies: [PhonePe](#) [DataBricks](#)

Problem Requirement

We need to find the length of the longest contiguous subarray (window) such that the absolute difference between any two elements in this subarray is less than or equal to the given `limit`.

$$\max(\text{window}) - \min(\text{window}) \leq \text{limit} \rightarrow \text{Valid Window}$$

Approach 1: Brute Force

Keep extending the window and recompute the maximum and minimum elements dynamically for each possible subarray.

```
for (int i = 0; i < n; i++) {
    int mn = nums[i];
    int mx = nums[i];

    for (int j = i; j < n; j++) {
        mn = min(mn, nums[j]);
        mx = max(mx, nums[j]);

        if (mx - mn <= limit) {
            ans = max(ans, j - i + 1);
        } else {
            break; // Difference will only increase or stay the same, as the culprit
                    // elements are still in the window
        }
    }
}
```

Time Complexity (TC): $O(N^2)$

Space Complexity (SC): $O(1)$

Approach 2: Multiset Approach

Use a balanced binary search tree (like `std::multiset` in C++) to automatically keep the elements in sorted order within the sliding window, allowing quick retrieval of the current minimum and maximum elements.

```
while (r < n) {
    st.insert(nums[r]);

    while (*st.rbegin() - *st.begin() > limit) {
        st.erase(st.find(nums[l]));
        l++;
    }
    ans = max(ans, r - l + 1);
    r++;
}
```

Time Complexity (TC): $O(N \log N)$

Space Complexity (SC): $O(N)$

Approach 3: Optimal Approach (Monotonic Deque)

Efficiently track the maximum and minimum elements using two companion double-ended queues (deques):

- **mxdq (Max-Deque):** Maintains elements in a decreasing order [*largest* → *smallest*]. While inserting `nums[r]`, remove all elements from the back that are smaller than it.
- **mindq (Min-Deque):** Maintains elements in an increasing order [*smallest* → *largest*]. While inserting `nums[r]`, remove all elements from the back that are greater than it.

Structure for Maintenance:

```
while (!mxdq.empty() && mxdq.back() < nums[r]) {
    mxdq.pop_back();
}
mxdq.push_back(nums[r]);

// Similarly do for mindq
```

Shrinking the Window:

```
while (mxdq.front() - mindq.front() > limit) {
    if (nums[l] == mxdq.front()) mxdq.pop_front();
    if (nums[l] == mindq.front()) mindq.pop_front();
    l++;
}
ans = max(ans, r - l + 1);
r++;
```

Time Complexity (TC): $O(N)$ — Each element is pushed and popped from the deques at most twice.

Space Complexity (SC): $O(N)$ — To store element indices/values inside the deques.

Complete Implementation

```
class Solution {
public:
    int longestSubarray(vector<int>& nums, int limit) {
        int n = nums.size();
        int r = 0;
        int l = 0;
        int ans = 0;
        deque<int> mxdq;
        deque<int> mindq;
```

```

while (r < n) {
    while (!mxdq.empty() && mxdq.back() < nums[r]) {
        mxdq.pop_back();
    }
    mxdq.push_back(nums[r]);

    while (!mndq.empty() && mndq.back() > nums[r]) {
        mndq.pop_back();
    }
    mndq.push_back(nums[r]);

    while (mxdq.front() - mndq.front() > limit) {
        if (nums[l] == mxdq.front()) mxdq.pop_front();
        if (nums[l] == mndq.front()) mndq.pop_front();
        l++;
    }

    ans = max(ans, r - l + 1);
    r++;
}

return ans;
}
};

```